Unity Graphics Programming - Volume 3

Contents

Preface - Unity Graphics Programming Volume 3

About This Book

This book is the third volume in the "Unity Graphics Programming" series, which explains graphics programming techniques using Unity.

In this series, the authors explore various topics driven by their interests, providing explanations ranging from introductory content for beginners to advanced tips for intermediate and expert users.

Source Code

The source code explained in each chapter is publicly available on the GitHub repository: **https://github.com/IndieVisualLa**

You can follow along with this book while running the code locally.

Reading Guide

The difficulty level varies by chapter, and depending on your knowledge level, some content may feel too simple or too challenging. We recommend reading articles on topics that interest you based on your current knowledge level.

For those who work with graphics programming professionally, we hope this book helps expand your repertoire of effects and techniques.

For students interested in visual coding who have experience with Processing or openFrameworks but still find 3D CG intimidating, we hope Unity serves as an accessible entry point to discover the expressive power of 3D CG and get started with development.

About IndieVisualLab

IndieVisualLab is a circle founded by colleagues (and former colleagues) from the same company. At work, we use Unity to create content programming for exhibition works in what is commonly called media art - utilizing Unity in ways that differ from typical game development.

Throughout this book, you may find knowledge that proves useful when applying Unity to exhibition works.

Recommended Environment

Some content in this book uses Compute Shaders, Geometry Shaders, and similar features. We recommend an environment that supports DirectX 11. However, some chapters are entirely CPU-side programs (C#) and don't require GPU features.

Due to environmental differences, the sample code behavior may not work correctly in all cases. Please report issues to the GitHub repository or adapt the code as needed.

Feedback and Requests

If you have feedback, concerns, or requests about this book (such as "I'd like to read an explanation about X"), please let us know through: - Web form: https://docs.google.com/forms/d/e/1FAIpQLSdxeansJvQGTWfZTBN_2RTuCK
- Email: lab.indievisual@gmail.com

Volume 3 Chapter Overview

1. **Baking Skinned Animation to Texture** - Pre-computing animation data for massive character rendering
2. **Gravitational N-Body Simulation** - GPU-based celestial body simulation
3. **Screen Space Fluid Rendering** - Deferred shading techniques for fluid visualization
4. **GPU-Based Cellular Growth Simulation** - Simulating cell division and growth on the GPU
5. **Reaction Diffusion** - Generating natural patterns using the Gray-Scott model
6. **Strange Attractor** - Visualizing chaotic mathematical systems
7. **Portal Implementation in Unity** - Recreating the Portal game mechanics
8. **Simple Soft Deformation** - Cartoon-style squash and stretch effects

Chapter 1: Baking Skinned Animation to Texture

Author: Sugino (@sugi_cho)

Introduction

This chapter demonstrates how to display thousands or tens of thousands of skinned animated objects efficiently. The technique involves baking animation vertex data into textures, enabling GPU instancing for massive character rendering.

When you want to express flocks of birds or crowds in Unity, you typically use Animator and SkinnedMeshRenderer components. However, SkinnedMeshRenderer doesn't support GPU instancing, meaning each object must be rendered individually - resulting in extremely heavy processing.

The solution presented here stores animated vertex position information as textures. This chapter explains the methodology, implementation considerations, and applications.

The Performance Problem

Testing with 5000 Animated Objects

Using a simple horse model with 1,890 vertices and animation, rendering 5,000 horses with standard SkinnedMeshRenderer results in approximately 8.8 FPS - extremely poor performance.

Profiler Analysis

Using Unity's Profiler (Window > Profiler or Ctrl+7), adding GPU Usage profiling reveals: - GPU processing time exceeds CPU processing time - CPU waits for GPU completion - Approximately 70% of GPU processing is consumed by `PostLateUpdate.UpdateAllSkinnedMeshes` - `Camera.Render` runs for each visible object

Key Insight: When GPU Skinning is enabled (Player Settings), the CPU calculates bone matrices and sends them to the GPU for skinning. With CPU skinning, both calculations happen on the CPU, resulting in even worse performance.

Optimization Principle: Always profile first to identify bottlenecks before optimizing.

Pre-Computing Skinned Mesh Vertices

Using SkinnedMeshRenderer.BakeMesh()

The `SkinnedMeshRenderer.BakeMesh(Mesh)` function creates a snapshot of the skinned mesh state and stores it in the specified mesh. While somewhat slow, it's suitable for pre-computing vertex information.

```
Animator animator;
SkinnedMeshRenderer skinedMesh;
List<Mesh> meshList;

void Start(){
    animator = GetComponent<Animator>();
    skinnedMesh = GetComponentInChildren<SkinnedMeshRenderer>();
    meshList = new List<Mesh>();
    animator.Play("Run");
}

void Update(){
    var mesh = new Mesh();
    skinnedMesh.BakeMesh(mesh);
    // mesh now contains the skinned mesh snapshot
    meshList.Add(mesh);
}
```

Why Store in Textures?

Storing full Mesh objects for each frame wastes memory on unchanging data (indices, UVs, etc.). For skinned animation, only vertex positions and normals change. Furthermore, updating mesh data per-frame via `Mesh.SetVertices()` creates significant CPU overhead.

The solution: Store position and normal data in textures and use **Vertex Texture Fetch (VTF)** in the vertex shader. This eliminates CPU overhead entirely.

Writing Position Data to Textures

Data Structure Mapping

Vector3 (Position)		Color (Texture)	
x	float	r	float
y	float	g	float
z	float	b	float

Implementation

```
public void CreateTex(Mesh sourceMesh)
{
    var vertCount = sourceMesh.vertexCount;
    var width = Mathf.FloorToInt(Mathf.Sqrt(vertCount));
    var height = Mathf.CeilToInt((float)vertCount / width);
    // Ensure vertCount < width * height

    // RGBAFloat format preserves full float precision
    posTex = new Texture2D(width, height, TextureFormat.RGBAFloat, false);
    normTex = new Texture2D(width, height, TextureFormat.RGBAFloat, false);

    var vertices = sourceMesh.vertices;
    var normals = sourceMesh.normals;
    var posColors = new Color[width * height];
    var normColors = new Color[width * height];
```

```
    for (var i = 0; i < vertCount; i++)
    {
        posColors[i] = new Color(
            vertices[i].x,
            vertices[i].y,
            vertices[i].z
        );
        normColors[i] = new Color(
            normals[i].x,
            normals[i].y,
            normals[i].z
        );
    }

    posTex.SetPixels(posColors);
    normTex.SetPixels(normColors);
    posTex.Apply();
    normTex.Apply();
}
```

Note: Although Unity documentation states `Texture2D.SetPixels()` only works with fixed-point formats (RGBA32, ARGB32, RGB24, Alpha8), it actually works with RGBAHalf and RGBAFloat, preserving negative values and values greater than 1 without clamping.

Complete Implementation

The system consists of three components:

1. **AnimationClipTextureBaker.cs** - Samples animation and creates ComputeBuffers
2. **MeshInfoTextureGen.compute** - Converts buffers to textures via ComputeShader
3. **TextureAnimPlayer.shader** - Plays animation from textures

Vertex Information Structure

```
public struct VertInfo
{
    public Vector3 position;
    public Vector3 normal;
}
```

Animation Sampling with AnimationClip.SampleAnimation()

```
// Sample animation at specific time without Animator component
clip.SampleAnimation(gameObject, time);
```

ComputeShader for Texture Generation

```
#pragma kernel CSMain

struct MeshInfo{
    float3 position;
    float3 normal;
};
```

```
RWTexture2D<float4> OutPosition;
RWTexture2D<float4> OutNormal;
StructuredBuffer<MeshInfo> Info;
int VertCount;

[numthreads(8,8,1)]
void CSMain (uint3 id : SV_DispatchThreadID)
{
    int index = id.y * VertCount + id.x;
    MeshInfo info = Info[index];

    OutPosition[id.xy] = float4(info.position, 1.0);
    OutNormal[id.xy] = float4(info.normal, 1.0);
    // X-axis = vertex ID, Y-axis = time
}
```

Texture Configuration

Critical Settings: - `FilterMode = Bilinear` - Enables automatic interpolation between frames - `Wrap-Mode = Repeat` - For looping animations (seamless loop) - `WrapMode = Clamp` - For non-looping animations

Bilinear filtering automatically interpolates between adjacent pixels, creating smooth animation even between sampled keyframes.

Animation Playback Shader

```
Shader "Unlit/TextureAnimPlayer"
{
    Properties
    {
        _MainTex ("Texture", 2D) = "white" {}
        _PosTex("position texture", 2D) = "black"{}
        _NmlTex("normal texture", 2D) = "white"{}
        _DT ("delta time", float) = 0
        _Length ("animation length", Float) = 1
        [Toggle(ANIM_LOOP)] _Loop("loop", Float) = 0
    }

    SubShader
    {
        Pass
        {
            CGPROGRAM
            #pragma vertex vert
            #pragma fragment frag
            #pragma multi_compile ___ ANIM_LOOP

            #include "UnityCG.cginc"
            #define ts _PosTex_TexelSize

            struct appdata
            {
                float2 uv : TEXCOORD0;
```

```
            };

            struct v2f
            {
                float2 uv : TEXCOORD0;
                float3 normal : TEXCOORD1;
                float4 vertex : SV_POSITION;
            };

            sampler2D _MainTex, _PosTex, _NmlTex;
            float4 _PosTex_TexelSize;
            float _Length, _DT;

            v2f vert (appdata v, uint vid : SV_VertexID)
            {
                float t = (_Time.y - _DT) / _Length;
                #if ANIM_LOOP
                    t = fmod(t, 1.0);
                #else
                    t = saturate(t);
                #endif

                // UV.x = vertex ID, UV.y = normalized time
                float x = (vid + 0.5) * ts.x;
                float y = t;

                float4 pos = tex2Dlod(_PosTex, float4(x, y, 0, 0));
                float3 normal = tex2Dlod(_NmlTex, float4(x, y, 0, 0));

                v2f o;
                o.vertex = UnityObjectToClipPos(pos);
                o.normal = UnityObjectToWorldNormal(normal);
                o.uv = v.uv;
                return o;
            }

            half4 frag (v2f i) : SV_Target
            {
                half diff = dot(i.normal, float3(0, 1, 0)) * 0.5 + 0.5;
                half4 col = tex2D(_MainTex, i.uv);
                return diff * col;
            }
            ENDCG
        }
    }
}
```

Key Concepts: - `SV_VertexID` semantic retrieves the vertex index - The `+0.5` offset ensures sampling at pixel centers (avoiding interpolation between vertices) - `_TexelSize` contains texture dimensions: x=1/width, y=1/height, z=width, w=height

Results

With texture-based animation, 5,000 horses render at **56.4 FPS** compared to 8.8 FPS with SkinnedMeshRenderer - a **6.4x improvement** without any additional optimizations like GPU instancing.

Since SkinnedMeshRenderer is no longer used, GPU instancing becomes possible, enabling even further performance gains with `Graphics.DrawMeshInstancedIndirect()`.

Limitations and Considerations

Constraints

1. **Memory Usage**: Texture size depends on vertex count and animation length
2. **Animation Blending**: Requires custom shader implementation
3. **State Machine**: Cannot use AnimatorController
4. **Maximum Texture Size**: Hardware-dependent (4K, 8K, or 16K)

Texture Size Limitation Workarounds

- Use multiple textures for high vertex counts
- Bake skeleton bone matrices instead of vertex positions (reduces data, enables runtime skinning in vertex shader)

Best Use Cases

This technique works best for: - Looping animations on background characters - Flocks of birds or butterflies - Crowd simulations - Any scenario where precise animation control is less critical

Key Takeaways

1. **Always profile first** - Identify bottlenecks before optimizing
2. **Skinned animation is expensive** - `UpdateAllSkinnedMeshes` often dominates GPU time
3. **Pre-baking to textures** eliminates runtime skinning cost
4. **GPU instancing becomes possible** when not using SkinnedMeshRenderer
5. **The technique generalizes** - Skeleton matrices, simulation results, or any expensive computation can be pre-baked to textures

Further Reading

- GPU Instancing documentation
- Vertex Texture Fetch (VTF) techniques
- Author's GitHub for advanced examples including instanced rendering

Chapter 2: Gravitational N-Body Simulation

Author: Takao

Introduction

This chapter explains how to implement a GPU-based Gravitational N-Body Simulation - a method for simulating the motion of celestial bodies in space.

Sample Project: `Assets/NBodySimulation` in the Unity Graphics Programming 3 repository.

What is N-Body Simulation?

N-Body simulation is a general term for simulations that calculate interactions between N physical objects. There are many types of problems that use N-Body simulation, and specifically, problems dealing with celestial bodies attracting each other through gravity in space are called **gravitational many-body problems**.

The algorithm explained in this chapter falls into this category - using N-Body simulation to solve the equations of motion for gravitational many-body systems.

Applications Beyond Gravity

N-Body simulation is applied across many fields: - Molecular force calculations - Dark matter analysis - Galaxy cluster collision analysis

From the smallest particles to cosmic scales, the technique has broad applications.

The Algorithm

Vector Form of Universal Gravitation

The basic universal gravitation equation from physics:

$$f = G\frac{Mm}{r^2}$$

This gives only the magnitude (scalar) of gravitational force between two bodies. For 3D simulation, we need the vector form.

The force vector that body *i* receives from body *j*:

$$\mathbf{f}_{ij} = G\frac{m_i m_j}{\|\mathbf{r}_{ij}\|^2} \cdot \frac{\mathbf{r}_{ij}}{\|\mathbf{r}_{ij}\|}$$

Where: - $\mathbf{f}_{ij}$ = force vector on body *i* from body *j* - m_i, m_j = masses of the two bodies - $\mathbf{r}_{ij}$ = direction vector from body *j* to body *i*

The left part calculates magnitude; the right part provides the unit direction vector.

Total Force on a Single Body

The total force $\mathbf{F}_i$ on body *i* from all other bodies:

$$\mathbf{F}_i = \sum_{j \in N} \mathbf{f}_{ij} = Gm_i \cdot \sum_{j \in N} \frac{m_j \mathbf{r}_{ij}}{\|\mathbf{r}_{ij}\|^3}$$

Softening Factor

To simplify simulation and handle collisions, we introduce a softening factor ε:

$$\mathbf{F}_i \simeq Gm_i \cdot \sum_{j \in N} \frac{m_j \mathbf{r}_{ij}}{(\|\mathbf{r}_{ij}\|^2 + \varepsilon)^{3/2}}$$

This prevents division by zero when particles are at the same position (including self-interaction, which yields 0).

Converting Force to Acceleration

Using Newton's second law $m\mathbf{a} = \mathbf{f}$:

$$\mathbf{a}_i = \frac{\mathbf{F}_i}{m_i}$$

Substituting into our equation:

$$\mathbf{a}_i \simeq G \cdot \sum_{j \in N} \frac{m_j \mathbf{r}_{ij}}{(\|\mathbf{r}_{ij}\|^2 + \varepsilon)^{3/2}}$$

This is our final equation for computing acceleration.

Finite Difference Method

Understanding Differentiation

The mathematical definition of differentiation:

$$\frac{df}{dt} = \lim_{\Delta t \to 0} \frac{f(t + \Delta t) - f(t)}{\Delta t}$$

In physics, position, velocity, and acceleration are related: - Velocity = derivative of position - Acceleration = derivative of velocity

Finite Differences for Computers

Computers cannot represent infinity, so we approximate with small finite Δt:

$$\frac{df}{dt} \simeq \frac{f(t + \Delta t) - f(t)}{\Delta t}$$

For position and velocity:

$$\frac{dx}{dt} \simeq \frac{x(t + \Delta t) - x(t)}{\Delta t} = v(t)$$

$$\frac{dv}{dt} \simeq \frac{v(t + \Delta t) - v(t)}{\Delta t} = a(t)$$

The Update Equation

Combining these:

$$x(t + \Delta t) = x(t) + v(t + \Delta t)\Delta t = x(t) + (v(t) + a(t)\Delta t)\Delta t$$

This means: **knowing current position, velocity, and acceleration, we can compute the position at time** $t + \Delta t$.

For real-time simulation, Δt is typically the frame time (1/60 second at 60fps).

Implementation

Data Structure

```
public struct Body
{
    public Vector3 position;
    public Vector3 velocity;
    public float mass;
}
```

Buffer Initialization

```
void InitBuffer()
{
    // Create Read/Write buffers to prevent race conditions
    bodyBuffers = new ComputeBuffer[2];

    bodyBuffers[READ] = new ComputeBuffer(numBodies,
        Marshal.SizeOf(typeof(Body)));

    bodyBuffers[WRITE] = new ComputeBuffer(numBodies,
        Marshal.SizeOf(typeof(Body)));
}
```

Initial Particle Distribution

```
void DistributeBodies()
{
    Random.InitState(seed);
    float scale = positionScale * Mathf.Max(1, numBodies / DEFAULT_PARTICLE_NUM);

    Body[] bodies = new Body[numBodies];

    int i = 0;
    while (i < numBodies)
    {
        // Random sampling within a sphere
        Vector3 pos = Random.insideUnitSphere;

        bodies[i].position = pos * scale;
        bodies[i].velocity = Vector3.zero;
        bodies[i].mass = Random.Range(0.1f, 1.0f);

        i++;
    }

    bodyBuffers[READ].SetData(bodies);
    bodyBuffers[WRITE].SetData(bodies);
}
```

Simulation Loop

```
void Update()
{
```

```
    // Set constants
    NBodyCS.SetFloat("_DeltaTime", Time.deltaTime);
    NBodyCS.SetFloat("_Damping", damping);
    NBodyCS.SetFloat("_SofteningSquared", softeningSquared);
    NBodyCS.SetInt("_NumBodies", numBodies);

    // Thread dimensions
    NBodyCS.SetVector("_ThreadDim",
        new Vector4(SIMULATION_BLOCK_SIZE, 1, 1, 0));

    NBodyCS.SetVector("_GroupDim",
        new Vector4(Mathf.CeilToInt(numBodies / SIMULATION_BLOCK_SIZE), 1, 1, 0));

    // Set buffers
    NBodyCS.SetBuffer(0, "_BodiesBufferRead", bodyBuffers[READ]);
    NBodyCS.SetBuffer(0, "_BodiesBufferWrite", bodyBuffers[WRITE]);

    // Execute
    NBodyCS.Dispatch(0,
        Mathf.CeilToInt(numBodies / SIMULATION_BLOCK_SIZE), 1, 1);

    // Swap buffers
    Swap(bodyBuffers);
}
```

Rendering

```
void OnRenderObject()
{
    particleRenderMat.SetPass(0);
    particleRenderMat.SetBuffer("_Particles", bodyBuffers[READ]);
    Graphics.DrawProcedural(MeshTopology.Points, numBodies);
}
```

GPU Implementation with Shared Memory

N-Body simulation requires calculating interactions between all particles, resulting in $O(n^2)$ complexity. Using shared memory (from Unity Graphics Programming Vol.1, Chapter 3) dramatically improves performance.

Tiled Approach Concept

Particles within the same thread block share data via shared memory, reducing global memory I/O. The concept:

1. Each row represents a global thread (DispatchThreadID)
2. Each column represents particles being examined
3. Tiles process blocks of particles at a time
4. All rows run in parallel but synchronize within tiles

With 256 threads per block (SIMULATION_BLOCK_SIZE), each tile is 256x256.

Compute Shader Constants

```
#include "Body.cginc"

cbuffer cb {
    float _SofteningSquared, _DeltaTime, _Damping;
    uint _NumBodies;
    float4 _GroupDim, _ThreadDim;
};

StructuredBuffer<Body> _BodiesBufferRead;
RWStructuredBuffer<Body> _BodiesBufferWrite;

// Shared memory for the thread block
groupshared Body sharedBody[SIMULATION_BLOCK_SIZE];
```

Tile Implementation

```
float3 computeBodyForce(Body body, uint3 groupID, uint3 threadID)
{
    uint start = 0;
    uint finish = _NumBodies;

    float3 acc = (float3)0;
    int currentTile = 0;

    // Process each tile (block)
    for (uint i = start; i < finish; i += SIMULATION_BLOCK_SIZE)
    {
        // Load into shared memory
        sharedBody[threadID.x]
            = _BodiesBufferRead[wrap(groupID.x + currentTile, _GroupDim.x)
                * SIMULATION_BLOCK_SIZE + threadID.x];

        // Synchronize within group
        GroupMemoryBarrierWithGroupSync();

        // Calculate gravitational effects
        acc = gravitation(body, acc, threadID);

        GroupMemoryBarrierWithGroupSync();

        currentTile++;
    }

    return acc;
}
```

Gravitational Interaction

```
float3 gravitation(Body body, float3 accel, uint3 threadID)
{
    // Process all bodies in the tile
    for (uint i = 0; i < SIMULATION_BLOCK_SIZE;)
```

```
    {
        accel = bodyBodyInteraction(accel, sharedBody[i], body);
        i++;
    }
    return accel;
}

// Core gravitational calculation
float3 bodyBodyInteraction(float3 acc, Body b_i, Body b_j)
{
    float3 r = b_i.position - b_j.position;

    // distSqr = dot(r_ij, r_ij) + EPS^2
    float distSqr = r.x * r.x + r.y * r.y + r.z * r.z;
    distSqr += _SofteningSquared;

    // invDistCube = 1/distSqr^(3/2)
    float distSixth = distSqr * distSqr * distSqr;
    float invDistCube = 1.0f / sqrt(distSixth);

    // s = m_j * invDistCube
    float s = b_j.mass * invDistCube;

    // a_i = a_i + s * r_ij
    acc += r * s;

    return acc;
}
```

Position Update (Main Kernel)

```
[numthreads(SIMULATION_BLOCK_SIZE,1,1)]
void CSMain (
    uint3 groupID : SV_GroupID,
    uint3 threadID : SV_GroupThreadID,
    uint3 DTid : SV_DispatchThreadID
) {
    uint index = DTid.x;
    Body body = _BodiesBufferRead[index];

    float3 force = computeBodyForce(body, groupID, threadID);

    body.velocity += force * _DeltaTime;
    body.velocity *= _Damping;

    // Finite difference update
    body.position += body.velocity * _DeltaTime;

    _BodiesBufferWrite[index] = body;
}
```

Billboard Particle Rendering

What is a Billboard?

A billboard is a simple plane object that always faces the camera. Most particle systems use billboards for rendering.

Implementation with View Matrix

The view matrix contains camera position and rotation information. To make a billboard face the camera, we apply the **inverse of the view matrix's rotation component** as the model transformation.

Geometry Shader for Quad Expansion

```
[maxvertexcount(4)]
void geom(point v2g input[1], inout TriangleStream<g2f> outStream) {
    g2f o;
    float4 pos = input[0].pos;

    float4x4 billboardMatrix = UNITY_MATRIX_V;

    // Extract rotation only (zero out translation)
    billboardMatrix._m03 = billboardMatrix._m13 =
        billboardMatrix._m23 = billboardMatrix._m33 = 0;

    for (int x = 0; x < 2; x++) {
        for (int y = 0; y < 2; y++) {
            float2 uv = float2(x, y);
            o.uv = uv;

            o.pos = pos
                + mul(transpose(billboardMatrix), float4((uv * 2 - float2(1, 1))
                * _Scale, 0, 1));

            o.pos = mul(UNITY_MATRIX_VP, o.pos);
            o.id = input[0].id;

            outStream.Append(o);
        }
    }
    outStream.RestartStrip();
}
```

Making It Visually Interesting

The basic simulation tends to collapse all particles to the center, which isn't visually appealing. A simple trick: **truncate the interaction calculation early**:

```
float3 computeBodyForce(Body body, uint3 groupID, uint3 threadID)
{
    ...
    uint finish = _NumBodies / div;  // Cut off early
    ...
}
```

By not computing all interactions, particles form multiple clusters that interact with each other, creating more dynamic and interesting motion.

Key Takeaways

1. **N-Body simulation** calculates gravitational interactions between all particles
2. **Finite difference method** discretizes differential equations for computer simulation
3. **Shared memory optimization** is essential for $O(n^2)$ algorithms on GPU
4. **Tile-based processing** reduces global memory access
5. **Billboard rendering** efficiently displays particles as camera-facing quads
6. **Artistic liberties** (like truncating calculations) can improve visual results
7. The technique has applications from **molecular simulation to galaxy formation**

References

- GPU Gems 3 - Chapter 31: Fast N-Body Simulation with CUDA
- N-Body Simulation Research (Kagoshima University)
- Quaternions and Billboards (wgld.org)

Chapter 3: Screen Space Fluid Rendering

Author: Oishi

Introduction

This chapter introduces **Screen Space Fluid Rendering** using **Deferred Shading** as a technique for rendering particles as fluid-like surfaces.

What is Screen Space Fluid Rendering?

Traditional methods for rendering continuous bodies like fluids use techniques like Marching Cubes, but these are computationally expensive and not ideal for real-time applications requiring fine detail.

Screen Space Fluid Rendering was developed as a faster alternative for particle-based fluids. The core idea: generate surfaces from the depth of particles as seen from the camera in screen space.

Deferred Rendering Overview

Concept

Deferred Rendering performs **shading calculations in 2D screen space** rather than during geometry processing. Traditional rendering is called **Forward Rendering** for distinction.

Pipeline Comparison

Forward Rendering: - First pass: Lighting and shading during geometry processing

Deferred Rendering: - First pass (G-Buffer pass): Generate 2D images of normals, positions, depth, diffuse color - Second pass: Perform lighting/shading using G-Buffer data - The “deferred” name comes from actual rendering being delayed to later passes

Advantages

- Supports many light sources efficiently
- Only calculates shading for visible pixels (minimizes fragment shader executions)
- Enables geometry modification
- G-Buffer data available for post-effects (SSAO, etc.)

Disadvantages

- Weak at semi-transparent rendering
- Some anti-aliasing algorithms (MSAA) don't work well
- Difficult to use multiple materials
- No support for orthographic cameras

Unity Requirements

- Unity Pro only
- Multiple Render Targets (MRT) support
- Shader Model 3.0+
- Graphics card supporting depth render textures and two-sided stencil buffers

G-Buffer (Geometry Buffer)

The G-Buffer stores screen-space 2D texture information for shading: normals, positions, diffuse reflectance, etc.

Default Unity G-Buffer Structure

Render Target	Format	Data Type
RT0	ARGB32	Diffuse color (RGB), Occlusion (A)
RT1	ARGB32	Specular color (RGB), Roughness (A)
RT2	ARGB2101010	World space normal (RGB)
RT3	ARGB2101010	Emission + lighting
Z-buffer	-	Depth + Stencil

Accessing G-Buffer in Shaders

Shader Property Name	Data Type
_CameraGBufferTexture0	Diffuse color (RGB), occlusion (A)
_CameraGBufferTexture1	Specular color (RGB)
_CameraGBufferTexture2	World space normal (RGB)
_CameraGBufferTexture3	Emission + lighting
_CameraDepthTexture	Depth + Stencil

CommandBuffer

CPU scripts don't perform actual drawing - they add rendering commands to a **graphics command buffer** that the GPU reads and executes.

Unity's `CommandBuffer` API lets you insert command lists at specific points in the rendering pipeline, extending Unity's built-in pipeline.

Key methods include `Graphics.DrawMesh()` and `Graphics.DrawProcedural()`.

Coordinate Systems and Transformations

Understanding coordinate transformations is essential for screen-space calculations.

Homogeneous Coordinates

3D position vectors (x,y,z) are represented as 4D (x,y,z,w) for efficient matrix multiplication. Transform: (x/w, y/w, z/w, 1) = (x, y, z, w)

Coordinate Spaces

1. **Object Space** (Local/Model): Object-centered coordinates
2. **World Space** (Global): Scene-centered coordinates
3. **Eye/View Space** (Camera): Camera-centered coordinates
4. **Clip Space**: After projection, used for clipping
5. **Normalized Device Coordinates (NDC)**: -1 to 1 range after perspective divide
6. **Screen/Window Space**: Final pixel coordinates

Transformation Flow

Object Space □ (Model Transform) □ World Space □ (View Transform) □ Eye Space □ (Projection Transform) □ Clip Space □ (Perspective Divide) □ NDC □ (Viewport Transform) □ Screen Space

Implementation

Algorithm Overview

1. Render particles to generate screen-space depth image
2. Apply blur to smooth the depth
3. Calculate surface normals from depth
4. Calculate surface shading

Script Structure

Script	Function
ScreenSpaceFluidRenderer.cs	Main controller
RenderParticleDepth.shader	Calculate screen-space particle depth
BilateralFilterBlur.shader	Depth-attenuating blur effect
CalcNormal.shader	Calculate normals from depth
RenderGBuffer.shader	Write depth, normals, color to G-Buffer

Setting Up CommandBuffer

```
void OnWillRenderObject()
{
    var act = gameObject.activeInHierarchy && enabled;
    if (!act)
    {
        CleanUp();
        return;
    }

    var cam = Camera.current;
```

```
    if (!cam) return;

    if (!_cameras.ContainsKey(cam))
    {
        var buf = new CmdBufferInfo();
        buf.pass = CameraEvent.BeforeGBuffer;
        buf.buffer = new CommandBuffer();
        buf.name = "Screen Space Fluid Renderer";

        // Insert before G-Buffer generation
        cam.AddCommandBuffer(buf.pass, buf.buffer);
        _cameras.Add(cam, buf);
    }
}
```

The key is `CameraEvent.BeforeGBuffer` - inserting our custom rendering right before Unity generates the standard G-Buffer.

Step 1: Generate Particle Depth Image

```
// Get shader property ID
int depthBufferId = Shader.PropertyToID("_DepthBuffer");

// Create temporary RenderTexture
buf.GetTemporaryRT(depthBufferId, -1, -1, 24,
    FilterMode.Point, RenderTextureFormat.RFloat);

// Set render targets
buf.SetRenderTarget(
    new RenderTargetIdentifier(depthBufferId),
    new RenderTargetIdentifier(depthBufferId)
);
buf.ClearRenderTarget(true, true, Color.clear);

// Set particle data
_renderParticleDepthMaterial.SetFloat("_ParticleSize", _particleSize);
_renderParticleDepthMaterial.SetBuffer("_ParticleDataBuffer",
    _particleControllerScript.GetParticleDataBuffer());

// Draw particles as point sprites
buf.DrawProcedural(
    Matrix4x4.identity,
    _renderParticleDepthMaterial,
    0,
    MeshTopology.Points,
    _particleControllerScript.GetMaxParticleNum()
);
```

Point Sprite Generation (Geometry Shader)

```
static const float3 g_positions[4] =
{
    float3(-1, 1, 0),
    float3( 1, 1, 0),
```

```
    float3(-1,-1, 0),
    float3( 1,-1, 0),
};

[maxvertexcount(4)]
void geom(point v2g In[1], inout TriangleStream<g2f> SpriteStream)
{
    g2f o = (g2f)0;
    float3 vertpos = In[0].position.xyz;

    [unroll]
    for (int i = 0; i < 4; i++)
    {
        float3 pos = g_positions[i] * _ParticleSize;
        pos = mul(unity_CameraToWorld, pos) + vertpos;
        o.position = UnityObjectToClipPos(float4(pos, 1.0));
        o.uv = g_texcoords[i];
        o.vpos = UnityObjectToViewPos(float4(pos, 1.0)).xyz;
        o.size = _ParticleSize;

        SpriteStream.Append(o);
    }
    SpriteStream.RestartStrip();
}
```

Depth Calculation (Fragment Shader)

```
fragmentOut frag(g2f i)
{
    // Calculate hemisphere normal from UV
    float3 N = (float3)0;
    N.xy = i.uv.xy * 2.0 - 1.0;
    float radius_sq = dot(N.xy, N.xy);
    if (radius_sq > 1.0) discard;
    N.z = sqrt(1.0 - radius_sq);

    // Pixel position in clip space
    float4 pixelPos = float4(i.vpos.xyz + N * i.size, 1.0);
    float4 clipSpacePos = mul(UNITY_MATRIX_P, pixelPos);
    float depth = clipSpacePos.z / clipSpacePos.w;

    fragmentOut o = (fragmentOut)0;
    o.depthBuffer = depth;
    o.depthStencil = depth;

    return o;
}
```

Step 2: Blur the Depth Image

A bilateral filter blur smooths the depth while preserving edges, connecting adjacent particles into a continuous surface.

Step 3: Calculate Normals from Depth

Normals are calculated using **partial derivatives** in X and Y directions.

```
float3 uvToEye(float2 uv, float z)
{
    float2 xyPos = uv * 2.0 - 1.0;
    float4 clipPos = float4(xyPos.xy, z, 1.0);
    float4 viewPos = mul(unity_CameraInvProjection, clipPos);
    viewPos.xyz = viewPos.xyz / viewPos.w;
    return viewPos.xyz;
}

float3 getEyePos(float2 uv)
{
    return uvToEye(uv, sampleDepth(uv));
}

float4 frag(v2f_img i) : SV_Target
{
    float2 uv = i.uv.xy;
    float depth = tex2D(_DepthBuffer, uv);

    #if UNITY_REVERSED_Z
    if (Linear01Depth(depth) > 1.0 - 1e-3) discard;
    #else
    if (Linear01Depth(depth) < 1e-3) discard;
    #endif

    float2 ts = _DepthBuffer_TexelSize.xy;
    float3 posEye = getEyePos(uv);

    // Partial derivative in X
    float3 ddx = getEyePos(uv + float2(ts.x, 0.0)) - posEye;
    float3 ddx2 = posEye - getEyePos(uv - float2(ts.x, 0.0));
    ddx = abs(ddx.z) > abs(ddx2.z) ? ddx2 : ddx;

    // Partial derivative in Y
    float3 ddy = getEyePos(uv + float2(0.0, ts.y)) - posEye;
    float3 ddy2 = posEye - getEyePos(uv - float2(0.0, ts.y));
    ddy = abs(ddy.z) > abs(ddy2.z) ? ddy2 : ddy;

    // Cross product gives normal
    float3 N = normalize(cross(ddx, ddy));
    N = normalize(mul(_ViewMatrix, float4(N, 0.0)));

    return float4(N * 0.5 + 0.5, 1.0);
}
```

Step 4: Write to G-Buffer

```
buf.SetGlobalTexture("_NormalBuffer", normalBufferId);
buf.SetGlobalTexture("_DepthBuffer", depthBufferId);

_renderGBufferMaterial.SetColor("_Diffuse", _diffuse);
```

```
_renderGBufferMaterial.SetColor("_Specular",
    new Vector4(_specular.r, _specular.g, _specular.b, 1.0f - _roughness));
_renderGBufferMaterial.SetColor("_Emission", _emission);

// Set G-Buffer as render targets
buf.SetRenderTarget(
    new RenderTargetIdentifier[4]
    {
        BuiltinRenderTextureType.GBuffer0, // Diffuse
        BuiltinRenderTextureType.GBuffer1, // Specular + Roughness
        BuiltinRenderTextureType.GBuffer2, // World Normal
        BuiltinRenderTextureType.GBuffer3  // Emission
    },
    BuiltinRenderTextureType.CameraTarget  // Depth
);

buf.DrawMesh(quad, Matrix4x4.identity, _renderGBufferMaterial);
```

G-Buffer Output Structure

```
struct gbufferOut
{
    half4 diffuse  : SV_Target0;
    half4 specular : SV_Target1;
    half4 normal   : SV_Target2;
    half4 emission : SV_Target3;
    float depth    : SV_Depth;
};
```

Memory Cleanup

Always release temporary RenderTextures with `ReleaseTemporaryRT()` to prevent memory overflow.

Key Takeaways

1. **Screen Space Fluid Rendering** creates fluid surfaces from particle depth in screen space
2. **Deferred Rendering** enables screen-space geometry manipulation
3. **G-Buffer** stores geometry information for deferred shading calculations
4. **CommandBuffer** extends Unity's rendering pipeline at specific points
5. **Normal calculation from depth** uses partial derivatives (ddx/ddy)
6. Understanding **coordinate transformations** is essential for screen-space techniques

Limitations and Future Work

This sample focuses on deferred geometry manipulation. For realistic liquid rendering, additional features would include: - Light absorption based on thickness - Internal refraction - Background transparency - Caustics effects

References

- GDC: Screen Space Fluid Rendering for Games (Simon Green, NVIDIA)
- Real-time Rendering Fundamentals (Satoshi Kodaira)

Chapter 4: GPU-Based Cellular Growth Simulation

Author: Nakamura (@mattatz)

Introduction

This chapter develops a GPU-based program for simulating cell division and growth, inspired by the “Cell Division and Growth Algorithm 1” tutorial from iGeo, a procedural modeling library for Processing used in architecture.

Sample Project: `CellularGrowth` in the Unity Graphics Programming 3 repository.

Topics Covered

- Using Append/ConsumeStructuredBuffer for dynamic GPU object management
- Representing network structures on the GPU
- Sequential processing with atomic operations

Cell Division and Growth Simulation

The simulation uses two structures: **Particle** and **Edge**.

Particle Behaviors

Each Particle represents a cell with three behaviors:

1. **Growth**: Increases in size over time
2. **Repulsion**: Collides and repels other particles
3. **Division**: Splits into two particles under certain conditions

Edge Purpose

Edges represent cell adhesion. When particles divide, edges connect them like springs, pulling particles together to form network structures.

Implementation

Particle Structure

```
[StructLayout(LayoutKind.Sequential)]
public struct Particle_t {
    public Vector2 position;    // Position
    public Vector2 velocity;    // Velocity
    float radius;               // Current size
    float threshold;            // Maximum size
    int links;                  // Connected edge count
    uint alive;                 // Active flag
}
```

Append/ConsumeStructuredBuffer

These are LIFO (Last In First Out) containers available since DirectX 11, enabling dynamic object count management on the GPU.

- **AppendStructuredBuffer**: Adds data
- **ConsumeStructuredBuffer**: Retrieves data

Buffer Initialization

```
protected void Start() {
    // Particle buffer (ping-pong for read/write separation)
    particleBuffer = new PingPongBuffer(count, typeof(Particle_t));

    // Object pool buffer
    poolBuffer = new ComputeBuffer(
        count,
        Marshal.SizeOf(typeof(int)),
        ComputeBufferType.Append
    );
    poolBuffer.SetCounterValue(0);

    // Count buffer for tracking pool size
    countBuffer = new ComputeBuffer(
        4,
        Marshal.SizeOf(typeof(int)),
        ComputeBufferType.IndirectArguments
    );

    // Dividable particles pool
    dividablePoolBuffer = new ComputeBuffer(
        count,
        Marshal.SizeOf(typeof(int)),
        ComputeBufferType.Append
    );

    InitParticlesKernel();
}
```

Object Pool Pattern

The poolBuffer stores indices of inactive particles: 1. Initialize: Add all particle indices to pool (all inactive) 2. Spawn: Pop index from pool, activate that particle 3. Despawn: Push index back to pool, deactivate particle

Particle Initialization Kernel

```
THREAD
void InitParticles(uint3 id : SV_DispatchThreadID)
{
    uint idx = id.x;
    uint count, strides;
    _Particles.GetDimensions(count, strides);
    if (idx >= count) return;

    // Initialize particle as inactive
    Particle p = create();
    p.alive = false;
    _Particles[idx] = p;

    // Add index to object pool
    _ParticlePoolAppend.Append(idx);
```

```
}
```

Emitting Particles

```
protected void EmitParticlesKernel(Vector2 point, int emitCount = 32)
{
    // Prevent consuming from empty pool
    emitCount = Mathf.Max(0, Mathf.Min(emitCount, CopyPoolSize(poolBuffer)));
    if (emitCount <= 0) return;

    var kernel = compute.FindKernel("EmitParticles");
    compute.SetBuffer(kernel, "_Particles", particleBuffer.Read);
    compute.SetBuffer(kernel, "_ParticlePoolConsume", poolBuffer);
    compute.SetVector("_Point", point);
    compute.SetInt("_EmitCount", emitCount);

    Dispatch1D(kernel, emitCount);
}
```

Critical: Always check pool size before consuming to avoid undefined behavior.

```
THREAD
void EmitParticles(uint3 id : SV_DispatchThreadID)
{
    if (id.x >= (uint)_EmitCount) return;

    // Get inactive particle index from pool
    uint idx = _ParticlePoolConsume.Consume();

    Particle c = create();
    float2 offset = random_point_on_circle(id.xx + float2(0, _Time));
    c.position = _Point.xy + offset;
    c.radius = nrand(id.xx + float2(_Time, 0));

    _Particles[idx] = c;
}
```

Particle Update (Growth + Repulsion)

```
THREAD
void UpdateParticles(uint3 id : SV_DispatchThreadID)
{
    uint idx = id.x;
    uint count, strides;
    _ParticlesRead.GetDimensions(count, strides);
    if (idx >= count) return;

    Particle p = _ParticlesRead[idx];

    if (p.alive)
    {
        // Growth: increase size up to threshold
        p.radius = min(p.threshold, p.radius + _DT * _Grow);

        // Repulsion: check collision with all other particles
```

```
        for (uint i = 0; i < count; i++)
        {
            Particle other = _ParticlesRead[i];
            if (i == idx || !other.alive) continue;

            float2 dir = p.position - other.position;
            float l = length(dir);
            float r = (p.radius + other.radius) * _Repulsion;

            if (l < r)
            {
                p.velocity += normalize(dir) * (r - l);
            }
        }

        // Apply velocity with drag
        float2 vel = p.velocity * _DT;
        float vl = length(vel);
        if (vl > 0)
        {
            p.position += normalize(vel) * min(vl, _Limit);
            p.velocity = normalize(p.velocity) *
                min(length(p.velocity) * _Drag, _Limit);
        }
    }

    _Particles[idx] = p;
}
```

Ping-Pong Buffer Pattern

When threads read data modified by other threads (like collision detection), we need separate read and write buffers to prevent data races:

```
// Read from one buffer, write to another
compute.SetBuffer(kernel, "_ParticlesRead", particleBuffer.Read);
compute.SetBuffer(kernel, "_Particles", particleBuffer.Write);

Dispatch1D(kernel, count);

// Swap buffers for next frame
particleBuffer.Swap();
```

Division System

Division occurs periodically via coroutine:

```
protected IEnumerator IDivider()
{
    yield return 0;
    while(true)
    {
        yield return new WaitForSeconds(divideInterval);
        Divide();
    }
```

```
}

protected void Divide() {
    GetDividableParticlesKernel();
    DivideParticlesKernel(maxDivideCount);
}
```

Division Kernel

```
bool dividable_particle(Particle p, uint idx)
{
    // Divide when nearly full size
    float rate = (p.radius / p.threshold);
    return rate >= 0.95;
}

uint divide_particle(uint idx, float2 offset)
{
    Particle parent = _Particles[idx];
    Particle child = create();

    // Halve the size
    float rh = parent.radius * 0.5;
    parent.radius = child.radius = max(rh, 0.1);

    // Offset positions
    float2 center = parent.position;
    parent.position = center - offset;
    child.position = center + offset;

    // Random threshold for child
    child.threshold = rh * lerp(1.25, 2.0, nrand(float2(_Time, idx)));

    // Get child index from pool and store
    uint cidx = _ParticlePoolConsume.Consume();
    _Particles[cidx] = child;
    _Particles[idx] = parent;

    return cidx;
}
```

Network Structure with Edges

Edge Structure

```
[StructLayout(LayoutKind.Sequential)]
public struct Edge_t
{
    public int a, b;        // Connected particle indices
    public Vector2 force;   // Spring force
    uint alive;             // Active flag
}
```

Connecting Particles with Edges

```
void connect(int a, int b)
{
    uint eidx = _EdgePoolConsume.Consume();

    // Atomic increment of link counts
    InterlockedAdd(_Particles[a].links, 1);
    InterlockedAdd(_Particles[b].links, 1);

    Edge e;
    e.a = a;
    e.b = b;
    e.force = float2(0, 0);
    e.alive = true;
    _Edges[eidx] = e;
}
```

Atomic Operations

InterlockedAdd prevents race conditions when multiple threads modify the same memory location. It guarantees that read-modify-write operations complete atomically without interference from other threads.

```
// Safe increment across threads
InterlockedAdd(_Particles[a].links, 1);
```

Closed Network Division Pattern

Creates triangular networks:

```
void divide_edge_closed(uint idx)
{
    Edge e = _Edges[idx];
    Particle pa = _Particles[e.a];
    Particle pb = _Particles[e.b];

    if ((pa.links == 1) || (pb.links == 1))
    {
        // Create triangle: divide and connect to both endpoints
        uint cidx = divide_particle(e.a);
        connect(e.a, cidx);
        connect(cidx, e.b);
    }
    else
    {
        // Insert child between parent and neighbor
        float2 dir = pb.position - pa.position;
        float2 offset = normalize(dir) * pa.radius * 0.25;
        uint cidx = divide_particle(e.a, offset);

        connect(e.a, cidx);

        // Convert existing edge to connect child
        InterlockedAdd(_Particles[e.a].links, -1);
        InterlockedAdd(_Particles[cidx].links, 1);
```

```
        e.a = cidx;
    }

    _Edges[idx] = e;
}
```

Spring Forces Between Connected Particles

```
THREAD
void UpdateEdges(uint3 id : SV_DispatchThreadID)
{
    uint idx = id.x;
    Edge e = _Edges[idx];
    e.force = float2(0, 0);

    if (!e.alive) { _Edges[idx] = e; return; }

    Particle pa = _Particles[e.a];
    Particle pb = _Particles[e.b];

    if (!pa.alive || !pb.alive) { _Edges[idx] = e; return; }

    // Calculate spring force
    float2 dir = pa.position - pb.position;
    float r = pa.radius + pb.radius;
    float len = length(dir);

    if (abs(len - r) > 0)
    {
        float l = ((len - r) / r);
        e.force = normalize(dir) * l * _Spring;
    }

    _Edges[idx] = e;
}

THREAD
void SpringEdges(uint3 id : SV_DispatchThreadID)
{
    Particle p = _Particles[id.x];
    if (!p.alive || p.links <= 0) return;

    // More connections = weaker individual force
    float dif = 1.0 / p.links;

    // Find all connected edges and apply forces
    uint count, strides;
    _Edges.GetDimensions(count, strides);

    for (uint i = 0; i < count; i++)
    {
        Edge e = _Edges[i];
        if (!e.alive) continue;
```

```
        if (e.a == (int)id.x)
            p.velocity -= e.force * dif;
        else if (e.b == (int)id.x)
            p.velocity += e.force * dif;
    }

    _Particles[id.x] = p;
}
```

Branch Division Pattern

Creates tree-like structures:

```
void divide_edge_branch(uint idx)
{
    Edge e = _Edges[idx];
    Particle pa = _Particles[e.a];
    Particle pb = _Particles[e.b];

    // Divide from particle with fewer connections
    uint i = lerp(e.b, e.a, step(pa.links, pb.links));

    uint cidx = divide_particle(i);
    connect(i, cidx);
}
```

Adjusting `_MaxLink` (maximum connections per particle) dramatically changes branching behavior.

Key Takeaways

1. **Append/ConsumeStructuredBuffer** enables dynamic GPU object management
2. **Object pooling** on GPU uses index-based activation/deactivation
3. **Ping-pong buffers** prevent data races in parallel particle updates
4. **Atomic operations** (InterlockedAdd) ensure thread-safe counter updates
5. **Network structures** emerge from edge-connected particles
6. **Division patterns** create different growth behaviors (closed networks vs. branching)
7. The technique can extend to **3D** for organic mesh generation

Further Resources

- 3D extension: https://github.com/mattatz/CellularGrowth
- Andy Lomas: Morphogenetic Creations
- J.A. Kaandorp: Computational Biology
- Max Cooper music videos (using Houdini)

References

- iGeo Tutorial: http://igeo.jp/tutorial/55.html
- HLSL Atomic Functions: https://docs.microsoft.com/windows/desktop/direct3d11/direct3d-11-advanced-stages-cs-atomic-functions

Chapter 5: Reaction Diffusion

Author: kaiware007 (@kaiware007)

Introduction

Nature contains countless beautiful patterns - the stripes on tropical fish, the maze-like wrinkles on coral surfaces. The mathematical genius Alan Turing expressed the generation of such natural patterns through equations. Patterns generated by his equations are called **"Turing Patterns"**, and the equations are generally known as **reaction-diffusion equations**.

This chapter develops a Unity program using ComputeShaders to generate biological-like patterns based on these equations. We'll start with 2D implementation and include a 3D extension as a bonus.

Sample Project: `ReactionDiffusion` in the Unity Graphics Programming 3 repository.

For ComputeShader basics, see Unity Graphics Programming Vol.1, Chapter 2.

What is Reaction Diffusion?

Reaction Diffusion is a mathematical model describing how the concentration of one or more substances distributed in space changes due to two processes:

1. **Reaction**: Local chemical reactions where substances transform into each other
2. **Diffusion**: Spreading throughout the space

The Gray-Scott Model

This chapter uses the **Gray-Scott model**, published by P. Gray and S.K. Scott in 1983. The model uses two virtual substances (U and V) that fill a grid, react with each other, and diffuse to create various patterns over time.

How It Works

Reaction Process: 1. U is continuously **fed** into the space at a constant rate 2. When two V particles meet one U particle, they **react** to create another V 3. V is continuously **killed** (removed) at a constant rate to prevent infinite growth

Diffusion Process: - U and V spread to neighboring grid cells at different rates - The difference in diffusion speeds creates concentration differences, generating patterns

The Equations

$$\frac{\partial u}{\partial t} = D_u \Delta u - uv^2 + f(1-u)$$

$$\frac{\partial v}{\partial t} = D_v \Delta v + uv^2 - (f+k)v$$

Where: - **Diffusion terms** ($D_u \Delta u$, $D_v \Delta v$): The Laplacian (Δ) represents spreading to equalize concentration differences - **Reaction term** (uv^2): One U and two V react - U decreases, V increases - **Feed term** ($+f(1-u)$): U is replenished (more when U is low) - **Kill term** ($-(f+k)v$): V is removed at a constant rate

Unity Implementation

Sample Scene: `ReactionDiffusion2D_1`

Grid Data Structure

```
public struct RDData
{
    public float u; // U concentration
    public float v; // V concentration
}
```

Initialization

```
void Initialize()
{
    int wh = texWidth * texHeight;

    // Double buffering for read/write separation
    buffers = new ComputeBuffer[2];
    for (int i = 0; i < buffers.Length; i++)
    {
        buffers[i] = new ComputeBuffer(wh, Marshal.SizeOf(typeof(RDData)));
    }

    bufData = new RDData[wh];
    ResetBuffer();

    // Seed input buffer
    inputData = new Vector2[inputMax];
    inputBuffer = new ComputeBuffer(inputMax, Marshal.SizeOf(typeof(Vector2)));
}
```

Why double buffering? ComputeShaders run in parallel. When calculating based on neighboring cells, using a single buffer causes race conditions where some threads read already-updated values. Separate read/write buffers ensure consistency.

Update Processing

```
void UpdateBuffer()
{
    cs.SetInt("_TexWidth", texWidth);
    cs.SetInt("_TexHeight", texHeight);
    cs.SetFloat("_DU", du);
    cs.SetFloat("_DV", dv);
    cs.SetFloat("_Feed", feed);
    cs.SetFloat("_K", kill);

    cs.SetBuffer(kernelUpdate, "_BufferRead", buffers[0]);
    cs.SetBuffer(kernelUpdate, "_BufferWrite", buffers[1]);
    cs.Dispatch(kernelUpdate,
        Mathf.CeilToInt((float)texWidth / THREAD_NUM_X),
        Mathf.CeilToInt((float)texHeight / THREAD_NUM_X),
        1);

    SwapBuffer();
}
```

ComputeShader Update Kernel

```
[numthreads(THREAD_NUM_X, THREAD_NUM_X, 1)]
void Update(uint3 id : SV_DispatchThreadID)
{
    int idx = GetIndex(id.x, id.y);
    float u = _BufferRead[idx].u;
    float v = _BufferRead[idx].v;
    float uvv = u * v * v;

    _BufferWrite[idx].u = saturate(
        u + (_DU * LaplaceU(id.x, id.y) - uvv + _Feed * (1.0 - u))
    );
    _BufferWrite[idx].v = saturate(
        v + (_DV * LaplaceV(id.x, id.y) + uvv - (_K + _Feed) * v)
    );
}
```

Index Calculation (Wrapping)

```
int GetIndex(int x, int y) {
    x = (x < 0) ? x + _TexWidth : x;
    x = (x >= _TexWidth) ? x - _TexWidth : x;
    y = (y < 0) ? y + _TexHeight : y;
    y = (y >= _TexHeight) ? y - _TexHeight : y;
    return y * _TexWidth + x;
}
```

Laplacian Functions

The Laplacian samples the 8 surrounding cells, weighted to reduce diagonal influence:

```
float LaplaceU(int x, int y) {
    float sumU = 0;
    for (int i = 0; i < 9; i++) {
        int2 pos = laplaceIndex[i];
        int idx = GetIndex(x + pos.x, y + pos.y);
        sumU += _BufferRead[idx].u * laplacePower[i];
    }
    return sumU;
}

float LaplaceV(int x, int y) {
    float sumV = 0;
    for (int i = 0; i < 9; i++) {
        int2 pos = laplaceIndex[i];
        int idx = GetIndex(x + pos.x, y + pos.y);
        sumV += _BufferRead[idx].v * laplacePower[i];
    }
    return sumV;
}
```

Adding Seeds

Press ‘A’ for random seeds, ‘C’ for center seed:

```
[numthreads(THREAD_NUM_X, 1, 1)]
void AddSeed(uint id : SV_DispatchThreadID)
{
    if (_InputNum <= id) return;

    int w = _SeedSize;
    float radius = _SeedSize * 0.5;

    int centerX = _InputBufferRead[id].x;
    int centerY = _InputBufferRead[id].y;

    for (int x = 0; x < w; x++)
    {
        for (int y = 0; y < w; y++)
        {
            float dis = distance(
                float2(centerX, centerY),
                float2(centerX - w/2 + x, centerY - w/2 + y)
            );
            if (dis <= radius) {
                _BufferWrite[GetIndex((centerX + x), (centerY + y))].v = 1;
            }
        }
    }
}
```

RenderTexture Output

```
RenderTexture CreateRenderTexture(int width, int height)
{
    RenderTexture tex = new RenderTexture(width, height, 0,
        RenderTextureFormat.RFloat,
        RenderTextureReadWrite.Linear);
    tex.enableRandomWrite = true;
    tex.filterMode = FilterMode.Bilinear;
    tex.wrapMode = TextureWrapMode.Repeat;
    tex.Create();
    return tex;
}
```

RenderTextureFormat.RFloat: Single float per pixel, perfect for concentration difference.

Drawing to Texture

```
float GetValue(int x, int y) {
    int idx = GetIndex(x, y);
    float u = _BufferRead[idx].u;
    float v = _BufferRead[idx].v;
    return 1 - clamp(u - v, 0, 1);
}

[numthreads(THREAD_NUM_X, THREAD_NUM_X, 1)]
void Draw(uint3 id : SV_DispatchThreadID)
{
```

```
    float c = GetValue(id.x, id.y);
    _HeightMap[id.xy] = c;
}
```

Simple Rendering Shader

```
fixed4 frag (v2f i) : SV_Target
{
    fixed4 col = lerp(_Color0, _Color1, tex2D(_MainTex, i.uv).r);
    return col;
}
```

Parameter Variations

Small changes to Feed and Kill produce dramatically different patterns:

Coral-like Pattern

- Feed: 0.037, Kill: 0.06

Dotted Pattern

- Feed: 0.03, Kill: 0.062

Dividing/Vanishing Dots

- Feed: 0.0263, Kill: 0.06

Worm-like Pattern (straight lines avoiding collision)

- Feed: 0.077, Kill: 0.0615

Hole Pattern

- Feed: 0.039, Kill: 0.058

Chaotic/Unstable Pattern

- Feed: 0.026, Kill: 0.051

Ripple Pattern (continuously spreading waves)

- Feed: 0.014, Kill: 0.0477

Surface Shader Version

Sample Scene: `ReactionDiffusion2D_2`

Normal Map Generation

For 3D-looking results, generate a normal map from concentration differences:

```
heightMapTexture = CreateRenderTexture(texWidth, texHeight,
    RenderTextureFormat.RFloat);        // Height map
normalMapTexture = CreateRenderTexture(texWidth, texHeight,
    RenderTextureFormat.ARGBFloat);     // Normal map (needs XYZ)
```

Normal Calculation

```
float3 GetNormal(int x, int y) {
    float3 normal = float3(0, 0, 0);
    float c = GetValue(x, y);
    normal.x = ((GetValue(x - 1, y) - c) - (GetValue(x + 1, y) - c));
    normal.y = ((GetValue(x, y - 1) - c) - (GetValue(x, y + 1) - c));
    normal.z = 1;
    normal = normalize(normal) * 0.5 + 0.5;
    return normal;
}
```

Surface Shader Output

```
void surf(Input IN, inout SurfaceOutputStandard o) {
    float2 uv = IN.uv_MainTex;
    half v0 = tex2D(_MainTex, uv).x;
    float3 norm = UnpackNormal(tex2D(_NormalTex, uv));

    half p = smoothstep(_Threshold, _Threshold + _Fading, v0);

    o.Albedo = lerp(_Color0.rgb, _Color1.rgb, p);
    o.Alpha = lerp(_Color0.a, _Color1.a, p);
    o.Smoothness = lerp(_Smoothness0, _Smoothness1, p);
    o.Metallic = lerp(_Metallic0, _Metallic1, p);
    o.Normal = normalize(float3(norm.x, norm.y, 1 - _NormalStrength));
    o.Emission = lerp(_Emit0 * _EmitInt0, _Emit1 * _EmitInt1, p).rgb;
}
```

3D Extension

Sample Scene: ReactionDiffusion3D

3D RenderTexture

```
RenderTexture CreateTexture(int width, int height, int depth)
{
    RenderTexture tex = new RenderTexture(width, height, 0,
        RenderTextureFormat.RFloat, RenderTextureReadWrite.Linear);
    tex.volumeDepth = depth;
    tex.dimension = UnityEngine.Rendering.TextureDimension.Tex3D;
    tex.enableRandomWrite = true;
    tex.filterMode = FilterMode.Bilinear;
    tex.wrapMode = TextureWrapMode.Repeat;
    tex.Create();
```

```
    return tex;
}
```

3D Texture Declaration

```
RWTexture3D<float> _HeightMap;  // 3D texture
```

3D Laplacian (27 neighbors)

```
static const int3 laplaceIndex[27] = {
    int3(-1,-1,-1), int3(0,-1,-1), int3( 1,-1,-1),
    int3(-1, 0,-1), int3(0, 0,-1), int3(1, 0,-1),
    int3(-1, 1,-1), int3(0, 1,-1), int3(1, 1,-1),

    int3(-1,-1, 0), int3(0,-1, 0), int3(1,-1, 0),
    int3(-1, 0, 0), int3(0, 0, 0), int3(1, 0, 0),
    int3(-1, 1, 0), int3(0, 1, 0), int3(1, 1, 0),

    int3(-1,-1, 1), int3(0,-1, 1), int3(1,-1, 1),
    int3(-1, 0, 1), int3(0, 0, 1), int3(1, 0, 1),
    int3(-1, 1, 1), int3(0, 1, 1), int3(1, 1, 1),
};

static const float laplacePower[27] = {
    0.02, 0.02, 0.02,
    0.02, 0.1,  0.02,
    0.02, 0.02, 0.02,

    0.02, 0.1,  0.02,
    0.1, -1.0,  0.1,
    0.02, 0.1,  0.02,

    0.02, 0.02, 0.02,
    0.02, 0.1,  0.02,
    0.02, 0.02, 0.02
};
```

3D Rendering

3D volume textures can't be displayed with standard shaders. Options include: - **Marching Cubes** (used in the sample - see Unity Graphics Programming Vol.1, Chapter 7) - **Ray Marching / Volume Rendering** (see Hecomi's blog for excellent examples)

Key Takeaways

1. **Reaction-Diffusion** generates organic patterns from simple mathematical rules
2. **Gray-Scott model** uses Feed and Kill parameters to control pattern variety
3. **Double buffering** prevents race conditions in parallel neighbor calculations
4. **Laplacian operator** enables diffusion by averaging neighboring concentrations
5. **Small parameter changes** produce dramatically different patterns
6. **Normal maps** add 3D depth to 2D results
7. The technique extends to **3D volumes** with 27-neighbor Laplacians

Creative Applications

- Nakama Kouhei: “DIFFUSION” (https://vimeo.com/145251635)
- Kitahara Nobutaka: “Reaction-Diffusion” (https://vimeo.com/176261480)

Warning: Tweaking parameters can be addictive - time flies quickly!

References

- Reaction-Diffusion Tutorial: http://www.karlsims.com/rd.html
- Gray-Scott Model Demo: https://pmneila.github.io/jsexp/grayscott/

Chapter 6: Strange Attractor

Author: Sakota

Introduction

This chapter visualizes **Strange Attractors** - phenomena where states governed by specific differential or difference equations exhibit nonlinear chaotic behavior - using Unity and GPU computation.

Sample Project: `StrangeAttractors` in the Unity Graphics Programming 3 repository.

Requirements

- ComputeShader support (Shader Model 5.0)
- Tested on Unity 2018.2.9f1

What is a Strange Attractor?

In dissipative systems (non-equilibrium systems with energy input and output), states that maintain stable orbits over time are called **Attractors**.

Among these, attractors where small differences in initial conditions amplify over time, exhibiting chaotic behavior, are called **Strange Attractors**.

This chapter covers two examples: 1. **Lorenz Attractor** 2. **Thomas’ Cyclically Symmetric Attractor**

Lorenz Attractor

The Butterfly Effect

The term “butterfly effect” originated from meteorologist Edward N. Lorenz’s 1972 lecture titled “Does the Flap of a Butterfly’s Wings in Brazil Set Off a Tornado in Texas?”

This phrase describes how tiny initial differences don’t necessarily lead to similar results mathematically - they can amplify chaotically into unpredictable behavior.

The mathematical properties behind this observation were published by Lorenz in 1963 as the **Lorenz Attractor**.

Lorenz Equations

The Lorenz system is described by these nonlinear ordinary differential equations:

$$\frac{dx}{dt} = -px + py$$

$$\frac{dy}{dt} = -xz + rx - y$$

$$\frac{dz}{dt} = xy - bz$$

When parameters are set to **p=10, r=28, b=8/3**, the system exhibits chaotic Strange Attractor behavior.

Implementation

Particle Structure

```
protected struct Params
{
    Vector3 emitPos;
    Vector3 position;
    Vector3 velocity;
    float   life;
    Vector2 size;     // x = current, y = target
    Vector4 color;

    public Params(Vector3 emitPos, float size, Color color)
    {
        this.emitPos = emitPos;
        this.position = Vector3.zero;
        this.velocity = Vector3.zero;
        this.life = 0;
        this.size = new Vector2(0, size);
        this.color = color;
    }
}
```

This structure is defined in the abstract `StrangeAttractor.cs` class for reuse across different attractors.

Buffer Initialization

```
protected sealed override void InitializeComputeBuffer()
{
    if (cBuffer != null) cBuffer.Release();

    cBuffer = new ComputeBuffer(instanceCount, Marshal.SizeOf(typeof(Params)));
    Params[] parameters = new Params[cBuffer.count];

    for (int i = 0; i < instanceCount; i++)
    {
        var normalize = (float)i / instanceCount;
        var color = gradient.Evaluate(normalize);
        parameters[i] = new Params(
            Random.insideUnitSphere * emitterSize * normalize,
            particleSize,
```

```
            color
        );
    }
    cBuffer.SetData(parameters);
}
```

Particles are colored by ID using a gradient, creating beautiful trails as particles follow the attractor.

Parameter Setup

```
[SerializeField, Tooltip("Default is 10")]
float p = 10f;
[SerializeField, Tooltip("Default is 28")]
float r = 28f;
[SerializeField, Tooltip("Default is 8/3")]
float b = 2.666667f;

protected override void UpdateShaderUniforms()
{
    computeShaderInstance.SetFloat(pId, p);
    computeShaderInstance.SetFloat(rId, r);
    computeShaderInstance.SetFloat(bId, b);
}
```

Emit Kernel

```
#pragma kernel Emit
#pragma kernel Iterator

#define THREAD_X 128
#define THREAD_Y 1
#define THREAD_Z 1
#define DT 0.022

struct Params
{
    float3 emitPos;
    float3 position;
    float3 velocity;
    float  life;
    float2 size;
    float4 color;
};

RWStructuredBuffer<Params> buf;

[numthreads(THREAD_X, THREAD_Y, THREAD_Z)]
void Emit(uint id : SV_DispatchThreadID)
{
    Params p = buf[id];
    p.life = (float)id * -1e-05;  // Slight delay per particle
    p.position = p.emitPos;
    p.size.x = 0.0;  // Start invisible
    buf[id] = p;
}
```

The negative `life` value creates a staggered emission effect - particles don't all start at once, which: - Prevents all particles from following identical paths - Creates beautiful gradient trails as particles spread out

Iterator Kernel (Lorenz Equations)

```
#define DT 0.022

float p;
float r;
float b;

float3 LorenzAttractor(float3 pos)
{
    float dxdt = (p * (pos.y - pos.x));
    float dydt = (pos.x * (r - pos.z) - pos.y);
    float dzdt = (pos.x * pos.y - b * pos.z);
    return float3(dxdt, dydt, dzdt) * DT;
}

[numthreads(THREAD_X, THREAD_Y, THREAD_Z)]
void Iterator(uint id : SV_DispatchThreadID)
{
    Params p = buf[id];
    p.life.x += DT;

    // Size scales with velocity for natural appearance
    p.size.x = p.size.y * saturate(length(p.velocity));

    if (p.life.x > 0)
    {
        p.velocity = LorenzAttractor(p.position);
        p.position += p.velocity;
    }
    buf[id] = p;
}
```

Why Use Fixed Delta Time?

The code uses a constant `DT = 0.022` instead of Unity's `Time.deltaTime`. This is critical for two reasons:

1. **Frame rate independence**: When frame rate drops, `Time.deltaTime` becomes large, causing coarse, inaccurate calculations that distort the attractor shape.
2. **Numerical stability**: Strange Attractors are sensitive to time step size. Wrong values can cause the system to either collapse to a point or diverge to infinity.

Thomas' Cyclically Symmetric Attractor

Published by biologist Rene Thomas, this attractor is notable for: - Stable behavior regardless of initial conditions - Unique, visually interesting shape

Thomas' Equations

$$\frac{dx}{dt} = \sin y - bx$$

$$\frac{dy}{dt} = \sin z - by$$

$$\frac{dz}{dt} = \sin x - bz$$

When **b ≈ 0.208186**: Chaotic Strange Attractor behavior When **b ≈ 0**: Particles drift through space

Implementation

```
protected sealed override void InitializeComputeBuffer()
{
    if (cBuffer != null) cBuffer.Release();

    cBuffer = new ComputeBuffer(instanceCount, Marshal.SizeOf(typeof(Params)));
    Params[] parameters = new Params[cBuffer.count];

    for (int i = 0; i < instanceCount; i++)
    {
        var normalize = (float)i / instanceCount;
        var color = gradient.Evaluate(normalize);
        parameters[i] = new Params(
            Random.insideUnitSphere * emitterSize * normalize,
            particleSize,
            color
        );
    }
    cBuffer.SetData(parameters);
}
```

For visual appeal, particles are colored with a gradient from center to edge, creating a mantle-like appearance in the spherical initial distribution.

Thomas Attractor Kernel

```
float b;

float3 ThomasAttractor(float3 pos)
{
    float dxdt = -b * pos.x + sin(pos.y);
    float dydt = -b * pos.y + sin(pos.z);
    float dzdt = -b * pos.z + sin(pos.x);
    return float3(dxdt, dydt, dzdt) * DT;
}

[numthreads(THREAD_X, THREAD_Y, THREAD_Z)]
void Emit(uint id : SV_DispatchThreadID)
{
    Params p = buf[id];
```

```
    p.life = (float)id * -1e-05;
    p.position = p.emitPos;
    p.size.x = p.size.y;  // Start visible (unlike Lorenz)
    buf[id] = p;
}

[numthreads(THREAD_X, THREAD_Y, THREAD_Z)]
void Iterator(uint id : SV_DispatchThreadID)
{
    Params p = buf[id];
    p.life.x += DT;

    if (p.life.x > 0)
    {
        p.velocity = ThomasAttractor(p.position);
        p.position += p.velocity;
    }
    buf[id] = p;
}
```

Unlike the Lorenz implementation, particles start visible (`p.size.x = p.size.y`) because we want to see the initial spherical distribution transform into the attractor shape.

Other Strange Attractors

There are many other fascinating Strange Attractors to explore:

- **Ueda Attractor**: 2D motion (Kyoto University)
- **Aizawa Attractor**: Spinning motion
- **Rossler Attractor**: Spiral bands
- **Chen Attractor**: Double scroll
- **Halvorsen Attractor**: Three interlocked loops

Each has unique visual characteristics and mathematical properties.

Key Takeaways

1. **Strange Attractors** are chaotic systems that maintain stable orbits despite sensitivity to initial conditions
2. **Lorenz Attractor** (p=10, r=28, b=8/3) demonstrates the butterfly effect
3. **Thomas' Attractor** (b≈0.208186) creates unique symmetric patterns
4. **Fixed delta time** is essential for stable, consistent attractor shapes
5. **Staggered particle emission** creates beautiful gradient trails
6. **GPU implementation** enables real-time visualization of millions of particles
7. Strange Attractors offer **low computational cost** with **high visual impact**

Visual Techniques

- **Gradient coloring** by particle ID creates depth
- **Velocity-based sizing** makes particles appear to accelerate
- **Spherical initial distribution** shows dramatic transformation
- **Delayed emission** prevents uniform paths

References

- http://paulbourke.net/fractals/lorenz/
- https://en.wikipedia.org/wiki/Thomas%27_cyclically_symmetric_attractor
- Lorenz, E. N.: Deterministic Nonperiodic Flow, Journal of Atmospheric Sciences, Vol.20, pp.130-141, 1963
- Thomas, Rene (1999): “Deterministic chaos seen in terms of feedback circuits”
- http://www.algosome.com/articles/aizawa-attractor-chaos.html

Chapter 7: Portal Implementation in Unity

Author: Fuqunaga (@fuqunaga)

Introduction

Have you played Portal? Released by Valve in 2007, it’s a legendary puzzle action game featuring portals - holes connected like wormholes that let you see through to the other side, allowing objects and the player to warp through.

Essentially, it’s a “Anywhere Door” (Dokodemo Door). Using the portal gun, you can place portals on flat surfaces and navigate through them.

This chapter implements a simplified version of Portal’s mechanics in Unity.

Sample Project: `PortalGateSystem` in the Unity Graphics Programming 3 repository.

Project Overview

Required Elements

- Movable player character
- Field/environment
- Portal gun (spawns portals at target location)

Player Setup

The player needs a full body model (visible through portals) despite first-person view. The sample uses Unity’s **Adam** character model.

To prevent the player model from appearing in the main camera (causing polygon clipping issues), a **Player** layer is created and excluded from the main camera’s Culling Mask.

Controls

- Movement: WASD or arrow keys (Shift for sprint)
- Look: Mouse movement
- Portal fire: Left/Right mouse click
- Ball fire: E key
- Jump: Space

Field Setup

The sample uses **playGROWnd** assets from unity3d-jp for a Portal-like aesthetic. The field is a rectangular room with simplified collision using transparent colliders on each face (labeled **StageColl** layer).

Floor collision extends beyond the room to prevent objects from falling through after warping.

Gate Generation

Gates are oriented in local space with XY plane as the surface and Z+ as the pass-through direction.

Portal Gun Implementation

```
void Shot(int idx)
{
    RaycastHit hit;
    if (Physics.Raycast(transform.position,
        transform.forward,
        out hit,
        float.MaxValue,
        LayerMask.GetMask(new[] { "StageColl" })))
    {
        var gate = gatePair[idx];
        if (gate == null)
        {
            var go = Instantiate(gatePrefab);
            gate = gatePair[idx] = go.GetComponent<PortalGate>();

            var pair = gatePair[(idx + 1) % 2];
            if (pair != null)
            {
                gate.SetPair(pair);
                pair.SetPair(gate);
            }
        }

        gate.hitColl = hit.collider;

        var trans = gate.transform;
        var normal = hit.normal;
        var up = normal.y >= 0f ? transform.up : transform.forward;

        trans.position = hit.point + normal * gatePosOffset;
        trans.rotation = Quaternion.LookRotation(-normal, up);

        gate.Open();
    }
}
```

Key points: - Raycast only against **StageColl** layer - Store hit collider in `PortalGate.hitColl` for later collision handling - Position offset from surface to prevent z-fighting - **Up vector handling**: When placing on ceiling, using `transform.up` causes disorienting 180-degree flips; using `transform.forward` maintains intuitive orientation

Virtual Camera System

The Core Challenge

When a gate opens, you need to see through to the paired gate's location. The solution: **render to a RenderTexture using a Virtual Camera positioned at the paired gate, then display that texture on the portal surface**.

Camera Generation

```
private void OnWillRenderObject()
{
    // Called per camera
    VirtualCamera pairVC;
    if (!pairVCTable.TryGetValue(cam, out pairVC))
    {
        if ((vc == null) || vc.generation < maxGeneration)
        {
            pairVC = pairVCTable[cam] = CreateVirtualCamera(cam, vc);
            return;
        }
    }
}
```

Infinite Recursion Problem

When two gates face each other, they create a mirror effect - infinite recursion of gates within gates.

Each recursion level needs its own Virtual Camera: 1. Main camera sees Gate A needs VirtualCamera1 2. VirtualCamera1 sees Gate B needs VirtualCamera2 3. VirtualCamera2 sees Gate A needs VirtualCamera3 4. And so on…

Solution: Limit recursion with `maxGeneration` and use previous frame's texture for deeper levels.

Virtual Camera Creation

```
VirtualCamera CreateVirtualCamera(Camera parentCam, VirtualCamera parentVC)
{
    var rootCam = parentVC?.rootCamera ?? parentCam;
    var generation = parentVC?.generation + 1 ?? 1;

    var go = Instantiate(virtualCameraPrefab);
    go.name = rootCam.name + "_virtual" + generation;
    go.transform.SetParent(transform);

    var vc = go.GetComponent<VirtualCamera>();
    vc.rootCamera = rootCam;
    vc.parentCamera = parentCam;
    vc.parentGate = this;
    vc.generation = generation;

    vc.Init();

    return vc;
}
```

Camera Initialization

```
public void Init()
{
    camera_.aspect = rootCamera.aspect;
    camera_.fieldOfView = rootCamera.fieldOfView;
    camera_.nearClipPlane = rootCamera.nearClipPlane;
```

```
    camera_.farClipPlane = rootCamera.farClipPlane;
    camera_.cullingMask |= LayerMask.GetMask(new[] { PlayerLayerName });
    camera_.depth = parentCamera.depth - 1;

    camera_.targetTexture = tex0;
}
```

Note: **Player** layer is included in VirtualCamera's culling mask (unlike main camera) since the player should be visible through portals.

Camera depth is set lower than parent to ensure rendering happens first.

Camera Update

```
private void LateUpdate()
{
    if (parentCamera == null)
    {
        Destroy(gameObject);
        return;
    }

    camera_.enabled = parentGate.IsVisible(parentCamera);
    if (camera_.enabled)
    {
        var parentCamTrans = parentCamera.transform;
        parentGate.UpdateTransformOnPair(
            transform,
            parentCamTrans.position,
            parentCamTrans.rotation
        );

        UpdateCamera();
    }
}
```

Visibility Check

```
public bool IsVisible(Camera camera)
{
    var pos = transform.position;
    var camPos = camera.transform.position;

    // Direction check: is gate facing camera?
    var camToGateDir = (pos - camPos).normalized;
    var dot = Vector3.Dot(camToGateDir, transform.forward);
    if (dot > 0f)
    {
        // Frustum culling
        var planes = GeometryUtility.CalculateFrustumPlanes(camera);
        return GeometryUtility.TestPlanesAABB(planes, coll.bounds);
    }

    return false;
}
```

Two checks: 1. **Facing check**: Dot product between camera-to-gate direction and gate forward 2. **Frustum check**: Unity's built-in frustum-AABB intersection test

Transform Mapping to Paired Gate

```
public void UpdateTransformOnPair(Transform trans, Vector3 worldPos, Quaternion worldRot)
{
    // Convert to local space of this gate
    var localPos = transform.InverseTransformPoint(worldPos);
    var localRot = Quaternion.Inverse(transform.rotation) * worldRot;

    var pairGateTrans = pair.transform;
    var gateRot = pair.gateRot;  // 180-degree Y rotation

    // Convert from paired gate's local space to world
    var pos = pairGateTrans.TransformPoint(gateRot * localPos);
    var rot = pairGateTrans.rotation * gateRot * localRot;

    trans.SetPositionAndRotation(pos, rot);
}
```

The `gateRot` (180-degree Y rotation) flips the coordinate system from gate front to back.

Optimized Camera Parameters

```
void UpdateCamera()
{
    var pair = parentGate.pair;
    var pairTrans = pair.transform;
    var mesh = pair.GetComponent<MeshFilter>().sharedMesh;
    var vtxList = mesh.vertices
        .Select(vtx => pairTrans.TransformPoint(vtx)).ToList();

    // Minimize frustum to gate bounds
    TargetCameraUtility.Update(camera_, vtxList);

    // Oblique near clip plane at gate surface
    var clipPlane = CalcPlane(camera_,
        pairGateTrans.position,
        -pairGateTrans.forward);

    camera_.projectionMatrix = camera_.CalculateObliqueMatrix(clipPlane);
}
```

Two optimizations: 1. **Frustum narrowing**: Only render what's visible through the gate 2. **Oblique near clip**: Don't render anything between camera and gate surface

Gate Rendering

The shader handles multiple states: - Frame and interior display - Opening animation (expanding circle) - Pre-pairing state (wavy background) - Post-pairing fade to portal view - Fallback to previous frame when beyond max generation

Vertex Shader

```
v2f vert(appdata_img In)
{
    v2f o;

    float3 posWorld = mul(unity_ObjectToWorld, float4(In.vertex.xyz, 1)).xyz;
    float4 clipPos = mul(UNITY_MATRIX_VP, float4(posWorld, 1));
    float4 clipPosOnMain = mul(_MainCameraViewProj, float4(posWorld, 1));

    o.pos = clipPos;
    o.uv = In.texcoord;
    o.sposOnMain = ComputeScreenPos(clipPosOnMain);
    o.grabPos = ComputeGrabScreenPos(o.pos);
    return o;
}
```

Two screen positions: - `clipPos`: Current camera (for rendering) - `clipPosOnMain`: Main camera (for sampling VirtualCamera texture)

Fragment Shader

```
// Circle mask
float2 uv = (In.uv.xy - 0.5) * 2;
float insideRate = (1 - length(uv)) * _OpenRate;

// Wavy background (pre-pairing)
float4 grabUV = In.grabPos;
float2 grabOffset = float2(
    snoise(float3(uv, _Time.y)),
    snoise(float3(uv, _Time.y + 10))
);
grabUV.xy += grabOffset * 0.3 * insideRate;
float4 bgColor = tex2Dproj(_BackgroundTexture, grabUV);

// Portal view (post-pairing)
float2 sUV = In.sposOnMain.xy / In.sposOnMain.w;
float4 sideColor = tex2D(_MainTex, sUV);

// Blend based on connection state
float4 col = lerp(bgColor, sideColor, _ConnectRate);

// Frame rendering
float frame = smoothstep(0, 0.1, insideRate);
float frameColorRate = 1 - abs(frame - 0.5) * 2;
float mixRate = saturate(grabOffset.x + grabOffset.y);
float3 frameColor = lerp(_FrameColor0, _FrameColor1, mixRate);
col.xyz = lerp(col.xyz, frameColor, frameColorRate);

col.a = frame;
```

Object Warping

The **PortalObj** component handles warp-related behavior for any warping object.

Collision Disabling

Gates have large trigger colliders extending front and back. When objects enter, collision with the gate's surface is disabled.

```
private void OnTriggerStay(Collider other)
{
    var gate = other.GetComponent<PortalGate>();
    if ((gate != null) && !touchingGates.Contains(gate) && (gate.pair != null))
    {
        touchingGates.Add(gate);
        Physics.IgnoreCollision(gate.hitColl, collider_, true);
    }
}

private void OnTriggerExit(Collider other)
{
    var gate = other.GetComponent<PortalGate>();
    if (gate != null)
    {
        touchingGates.Remove(gate);
        Physics.IgnoreCollision(gate.hitColl, collider_, false);
    }
}
```

Note: Uses `OnTriggerStay` instead of `OnTriggerEnter` to handle cases where the object enters before the paired gate exists.

Warp Detection

```
private void Update()
{
    var passedGate = touchingGates.FirstOrDefault(gate =>
    {
        var posOnGate = gate.transform.InverseTransformPoint(center.position);
        return posOnGate.z > 0f;  // Behind the gate surface
    });

    if (passedGate != null)
    {
        PassGate(passedGate);
    }
}
```

Warp Execution

```
void PassGate(PortalGate gate)
{
    gate.UpdateTransformOnPair(transform);

    if (rigidbody_ != null)
    {
        rigidbody_.velocity = gate.UpdateDirOnPair(rigidbody_.velocity);
        rigidbody_.useGravity = false;
        ignoreGravityStartTime = Time.time;
```

```
    }

    if (fpController != null)
    {
        fpController.m_MoveDir = gate.UpdateDirOnPair(fpController.m_MoveDir);
        fpController.InitMouseLook();
    }
}
```

Gravity is temporarily disabled to prevent objects from oscillating between floor-to-floor portals.

Known Limitations

High-Speed Collision Issues

Fast-moving objects may collide with walls before the trigger system disables collision. The large trigger zone mitigates but doesn't fully solve this.

Wanted: Better method for disabling collision during the same physics frame as trigger entry.

Missing Partial Object Rendering

True portal behavior shows half an object on each side during transition. Current implementation teleports instantly.

Proper solution would require: - Duplicating objects during transition - Applying forces from both sides - Potentially interfering with physics solver internals

Key Takeaways

1. **Virtual Cameras** render portal views to RenderTextures
2. **Generation limiting** prevents infinite recursion with facing portals
3. **Oblique projection** clips at the gate surface for efficiency
4. **Coordinate transformation** maps positions between paired gates
5. **Trigger-based collision** handling enables pass-through
6. **GrabPass** provides background for pre-paired portals
7. Real portal systems require **careful physics handling**

Future Ideas

As rendering becomes more realistic, previously "unrealistic" ideas like portals gain new life. The "anywhere door" concept, long dismissed as fantasy, may prove surprisingly applicable to novel experiences.

References

- Portal: http://www.thinkwithportals.com/
- Adam Character Pack: Unity Asset Store
- playGROWnd: https://github.com/unity3d-jp/playgrownd
- PostProcessingStack: https://github.com/Unity-Technologies/PostProcessing

Chapter 8: Simple Soft Deformation

Author: xjine

Introduction

When expressing the softness of objects, you might simulate springs, fluids, or soft bodies. This chapter presents a simpler approach - achieving soft, cartoon-like deformation without complex physics calculations.

The technique creates hand-drawn animation-style squash and stretch effects.

Sample Project: `OverReaction` in the Unity Graphics Programming 3 repository.

Sample Scenes

OverReaction Scene

Basic deformation testing. Move objects using the manipulator or Inspector to see deformation.

PhysicsScene

Apply forces with arrow keys. Place multiple objects to observe context-dependent deformation.

Deformation Rules

Three fundamental principles govern deformation:

1. **Larger changes = larger deformation**
2. **No change = deformation gradually returns to normal**
3. **Reversed direction = reversed deformation**

We focus on movement-based deformation, tracking movement direction and magnitude as "move energy" (expressed as a vector).

Note: In proper physics terminology, kinetic energy is scalar. We use "move energy" as a directional quantity for this purpose.

Move Energy Calculation

Why FixedUpdate?

```
protected void FixedUpdate()
{
    this.crntMove = this.transform.position - this.prevPosition;

    UpdateMoveEnergy();
    UpdateDeformEnergy();
    DeformMesh();

    this.prevPosition = this.transform.position;
    this.prevMove = this.crntMove;
}
```

`FixedUpdate` runs at fixed time intervals, unlike `Update` which varies with frame rate. Benefits: - Consistent behavior with Unity's PhysX physics - No need for high-frequency mesh deformation - Predictable calculations regardless of frame rate

Component-Wise Energy Update

```
protected void UpdateMoveEnergy()
{
    this.moveEnergy = new Vector3()
    {
        x = UpdateMoveEnergy(
            this.crntMove.x, this.prevMove.x, this.moveEnergy.x),
        y = UpdateMoveEnergy(
            this.crntMove.y, this.prevMove.y, this.moveEnergy.y),
        z = UpdateMoveEnergy(
            this.crntMove.z, this.prevMove.z, this.moveEnergy.z),
    };
}
```

Each axis is calculated independently.

Sign Helper Function

```
public static int Sign(float value)
{
    return value == 0 ? 0 : (value > 0 ? 1 : -1);
}
```

Case 1: No Current Movement

When stationary, existing energy decays:

```
protected float UpdateMoveEnergy(float crntMove, float prevMove, float moveEnergy)
{
    int crntMoveSign = Sign(crntMove);
    int prevMoveSign = Sign(prevMove);
    int moveEnergySign = Sign(moveEnergy);

    if (crntMoveSign == 0)
    {
        return moveEnergy * this.undeformPower;  // Decay
    }
    // ...
}
```

Case 2: Direction Reversal (Current vs Previous)

When movement direction reverses, flip the energy:

```
if (crntMoveSign != prevMoveSign)
{
    return moveEnergy - crntMove;
}
```

Case 3: Energy Opposing Current Movement

When energy direction opposes movement, reduce energy:

```
if (crntMoveSign != moveEnergySign)
{
```

```
    return moveEnergy + crntMove;
}
```

Case 4: Same Direction

When energy and movement align, take the larger (decayed energy vs amplified movement):

```
if (crntMoveSign < 0)
{
    return Mathf.Min(crntMove * this.deformPower,
                     moveEnergy * this.undeformPower);
}
else
{
    return Mathf.Max(crntMove * this.deformPower,
                     moveEnergy * this.undeformPower);
}
```

- `deformPower`: Amplifies new movement for visible effect
- `undeformPower`: Decays existing energy (< 1.0)

Deform Energy Calculation

Deform energy determines actual mesh deformation, derived from move energy.

Energy Transfer Based on Direction

Not all energy transfers to deformation when movement and energy directions differ:

```
protected void UpdateDeformEnergy()
{
    float deformEnergyVertical
        = this.moveEnergy.magnitude
        * Vector3.Dot(this.moveEnergy.normalized,
                      this.crntMove.normalized);
    // ...
}
```

The dot product ranges: - **1.0**: Perfectly aligned directions (full transfer) - **0.0**: Perpendicular (no transfer) - **-1.0**: Opposite directions (negative/compression)

Horizontal Compensation

When an object stretches vertically, it should compress horizontally (and vice versa):

```
float deformEnergyHorizontalRatio
    = deformEnergyVertical / this.maxDeformScale;

float deformEnergyHorizontal
    = 1 - deformEnergyHorizontalRatio;
```

If vertical deformation is +0.8 (stretch), horizontal becomes 1 - 0.8 = 0.2 (compress).

Compression Case (Moving Into Energy)

When the object compresses in movement direction (negative dot product):

```
if (deformEnergyVertical < 0)
{
    deformEnergyVertical = deformEnergyHorizontalRatio;
}

deformEnergyVertical = 1 + deformEnergyVertical;
```

This inverts the stretch/compress relationship.

Complete UpdateDeformEnergy

```
protected void UpdateDeformEnergy()
{
    float deformEnergyVertical
        = this.moveEnergy.magnitude
        * Vector3.Dot(this.moveEnergy.normalized,
                      this.crntMove.normalized);

    float deformEnergyHorizontalRatio
        = deformEnergyVertical / this.maxDeformScale;

    float deformEnergyHorizontal
        = 1 - deformEnergyHorizontalRatio;

    if (deformEnergyVertical < 0)
    {
        deformEnergyVertical = deformEnergyHorizontalRatio;
    }

    deformEnergyVertical = 1 + deformEnergyVertical;

    // Clamp to valid range
    deformEnergyVertical = Mathf.Clamp(deformEnergyVertical,
                                       this.minDeformScale,
                                       this.maxDeformScale);

    deformEnergyHorizontal = Mathf.Clamp(deformEnergyHorizontal,
                                         this.minDeformScale,
                                         this.maxDeformScale);

    this.deformEnergy = new Vector3(deformEnergyHorizontal,
                                    deformEnergyVertical,
                                    deformEnergyHorizontal);
}
```

Mesh Deformation

The deform energy represents scaling along the movement direction. To apply it correctly, we need coordinate transformations.

Note: For production, consider GPU-based shader implementation for better performance.

Required Matrices

```
protected void DeformMesh()
{
    Vector3[] deformedVertices = new Vector3[this.baseVertices.Length];

    // Current object rotation and inverse
    Quaternion crntRotation = this.transform.localRotation;
    Quaternion crntRotationI = Quaternion.Inverse(crntRotation);

    // Move energy direction rotation and inverse
    Quaternion moveEnergyRotation
        = Quaternion.FromToRotation(Vector3.up, this.moveEnergy.normalized);
    Quaternion moveEnergyRotationI = Quaternion.Inverse(moveEnergyRotation);
    // ...
}
```

Transformation Steps

1. Apply current rotation to unrotated base vertices
2. Apply inverse move-energy rotation (align to up axis)
3. Scale by deformEnergy
4. Apply move-energy rotation (restore direction)
5. Apply inverse current rotation (restore orientation)

```
for (int i = 0; i < this.baseVertices.Length; i++)
{
    deformedVertices[i] = this.baseVertices[i];

    // 1. Apply current rotation
    deformedVertices[i] = crntRotation * deformedVertices[i];

    // 2. Rotate to align with up (for scaling)
    deformedVertices[i] = moveEnergyRotationI * deformedVertices[i];

    // 3. Apply deformation scaling
    deformedVertices[i] = new Vector3(
        deformedVertices[i].x * this.deformEnergy.x,
        deformedVertices[i].y * this.deformEnergy.y,
        deformedVertices[i].z * this.deformEnergy.z);

    // 4. Restore move-energy rotation
    deformedVertices[i] = moveEnergyRotation * deformedVertices[i];

    // 5. Remove current rotation
    deformedVertices[i] = crntRotationI * deformedVertices[i];
}

this.baseMesh.vertices = deformedVertices;
```

Debugging Tip: Comment out steps sequentially to understand each transformation's effect.

Key Takeaways

1. **Move energy** tracks object velocity as a persisting, decaying vector

2. **Deform energy** converts move energy to actual scale values
3. **Dot product** determines energy transfer based on direction alignment
4. **Volume preservation**: Vertical stretch causes horizontal compression
5. **Coordinate transformations** align deformation with movement direction
6. **FixedUpdate** ensures consistent physics-friendly calculations
7. Simple implementation yields **significant visual impact**

Possible Extensions

- Support for rotation-based deformation
- Scale change response
- Adjustable deformation center of mass
- Skinned mesh animation integration

Visual Impact

Despite the simple implementation, the technique dramatically changes visual impression. The cartoon-like squash and stretch brings objects to life with minimal computational cost.